A Cluster-Local Softmax Merge Unit for RISC-V Vector Clusters

Anonymous Author(s)

***Abstract*—Blockwise attention repeatedly merges a compact (m, l, O) state after each tile. This online merge combines a state-dependent scalar recurrence with a regular $O[D]$ vector update. In the strong RVV software baseline B2R, RVV handles the vector work while software executes the scalar dependence. We present Proposed, a Selective Offloading design that assigns the recurrence to a cluster-local Scalar Softmax Merge Unit (SMU) and reuses the existing RVV datapath after a TCDM-mediated weight handoff. RTL/Verilator measurements show that Proposed reduces the fitted per-row scalar term C_s by 17.66× over B2R while increasing the per-element vector term C_v by 1.88%. Across three model-derived configurations, Proposed achieves a 4.81× geometric-mean measured-cycle speedup over B2R. We define speedup as B2R cycles divided by Proposed cycles. The Full-Offload Ablation lowers C_s further but has 48.8% greater standalone mapped SMU area than Proposed. These results show that specializing the recurrence preserves efficient vector reuse while avoiding the cost of full offload.**

***Index Terms*—online softmax, attention, RISC-V vector processor, scratchpad, accelerator**

I. Introduction

Tiled attention avoids materializing the full score matrix by updating online-softmax state after each tile [1], [2]. Each merge produces m_{new}, l_{new}, w_{old}, and w_{tile} before the regular $O[D]$ vector update. The same pattern repeats for every row and tile in the kernel. This decomposition separates the state-dependent scalar recurrence from uniform elementwise work. The recurrence has row-level dependencies, whereas the vector update exposes element-level parallelism, so the two phases have different dependency/parallelism structures.

Spatz provides compact RVV execution and shared TCDM, which suits the regular elementwise update [3]. We use B2R as the strong software baseline: RVV executes the $O[D]$ update while software executes the scalar recurrence. Maximum selection, exponential rescaling, length accumulation, and weight generation remain serial within each row and tile. The Full-Offload Ablation lowers scalar cost but replaces the efficient existing RVV datapath with a specialized vector datapath. This contrast makes the specialization boundary the design question.

Our proposed design is exactly Scalar SMU + existing RVV. The Scalar SMU executes the state-dependent recurrence and writes m_{new}, l_{new}, w_{old}, and w_{tile} to TCDM. After completion, software loads the two weights from TCDM and invokes existing RVV for the regular $O[D]$ update. This implementation leaves the existing ISA and RVV datapath unchanged (Figure 1).

Across three model-derived configurations, Proposed achieves a 4.81× geometric-mean measured-cycle speedup over B2R, with speedup defined as B2R cycles divided by Proposed cycles. Relative to B2R, Proposed reduces the fitted per-row scalar term C_s by 17.66× while increasing the per-element vector term C_v by 1.88%; the Full-Offload Ablation lowers C_s further, but its C_v is 3.78× Proposed and its standalone mapped SMU area is 48.8% larger. We make three contributions:

- **Lightweight Scalar SMU.** We design a cluster-local accelerator for the state-dependent online-softmax recurrence, covering maximum selection, exponential rescaling, length accumulation, reciprocal normalization, and weight generation.
- **Selective Offloading.** We assign the scalar recurrence to the SMU and retain the regular $O[D]$ update on the existing RVV datapath through a post-completion, TCDM-mediated weight handoff.
- **Quantitative Validation.** We measure RTL/Verilator cycles across scaling and model-derived configurations and report standalone SMU mapped cells and Nangate45 Liberty cell-area units from Yosys/ABC.

II. Selective-Offload Architecture

Proposed is a Scalar SMU paired with existing RVV: the SMU handles the state-dependent scalar recurrence, while the core retains the regular $O[D]$ vector update. This selective boundary moves normalization control across the accelerator interface and leaves the established vector datapath and software sequencing in place. Shared TCDM carries the handoff state, so the architecture composes an RTL-defined scalar engine with an unchanged RVV update path. The scalar unit owns quantities whose next value depends on the row's selected maximum and normalization state; RVV owns operations that repeat over independent vector elements. The handoff therefore preserves software-visible memory ordering while narrowing hardware specialization to the recurrence. Figure 1 shows this selective-offload boundary and its TCDM-mediated handoff.

A. Specialization Boundary

Online merge separates state selection and normalization from the regular elementwise vector update. For one row, the

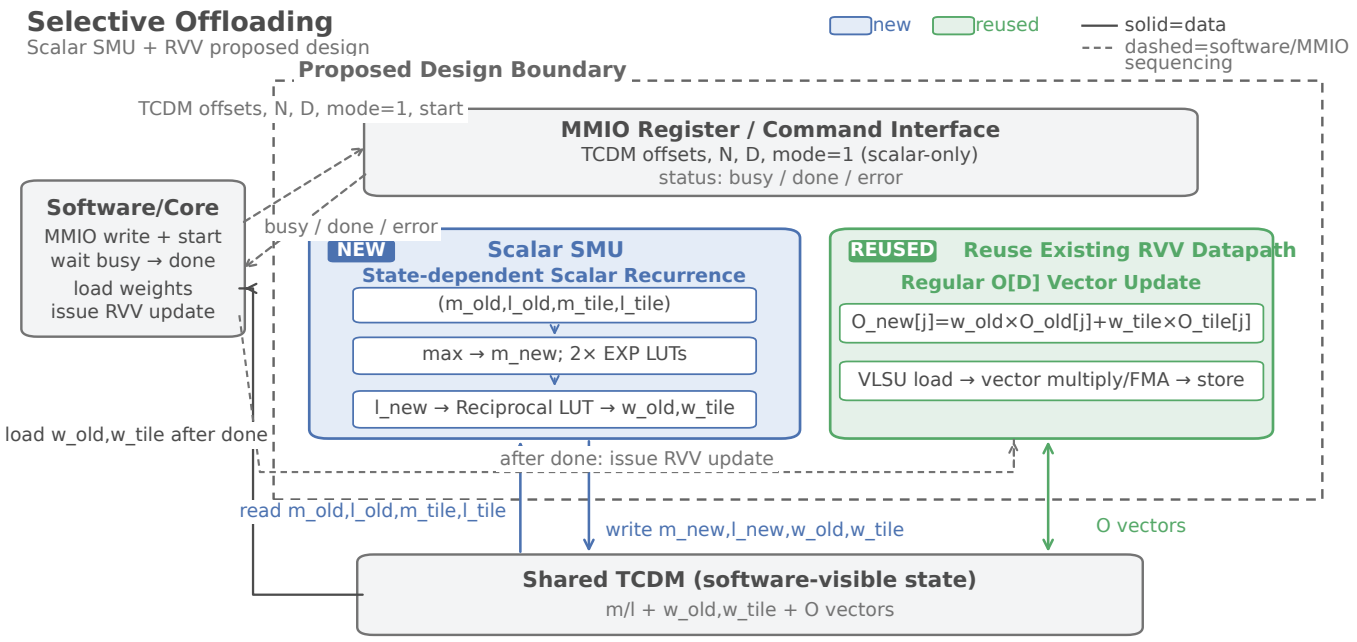

Fig. 1. Proposed architecture. Mode 1 directs the Scalar SMU to read scalar state and write m_{new}, l_{new}, w_{old}, and w_{tile} to shared TCDM. Software waits for completion, then invokes existing RVV for the $O[D]$ update; the diagram has no direct SMU-to-RVV link.

decomposition is

$$m_{\text{new}} = \max(m_{\text{old}}, m_{\text{tile}}), \tag{1}$$
$$s_{\text{old}} = l_{\text{old}} \exp(m_{\text{old}} - m_{\text{new}}), \tag{2}$$
$$s_{\text{tile}} = l_{\text{tile}} \exp(m_{\text{tile}} - m_{\text{new}}), \tag{3}$$
$$l_{\text{new}} = s_{\text{old}} + s_{\text{tile}}, \tag{4}$$
$$w_{\text{old}} = s_{\text{old}}/l_{\text{new}}, \quad w_{\text{tile}} = s_{\text{tile}}/l_{\text{new}}, \tag{5}$$
$$O_{\text{new}}[j] = w_{\text{old}} O_{\text{old}}[j] + w_{\text{tile}} O_{\text{tile}}[j], \quad 0 \le j < D. \tag{6}$$

The first five lines execute in Scalar SMU, while the final line executes in existing RVV. EXP and reciprocal blocks implement RTL approximations, so these equations describe the decomposition rather than an IEEE-exact arithmetic claim. The index constraint makes the handoff explicit: the SMU produces per-row weights and RVV applies them across D elements. These lines reduce each row to two weights before any vector element moves. The decomposition also separates row-level dependencies from element-level accumulation, which defines the architecture's scheduling boundary.

Scalar recurrence belongs on a state-dependent datapath because both exponential arguments depend on the selected maximum and the reciprocal depends on the newly accumulated length. A regular vector loop excels at independent $O[j]$ operations after weights are known, but it does not remove the serial decisions that produce those weights. The boundary therefore assigns control-sensitive work to Scalar SMU and keeps uniform multiply-add work on existing RVV. This division avoids duplicating vector functional units while matching each phase to its natural execution granularity. Both decisions serialize within a row, whereas elements within one O row share the resulting weights and expose regular parallelism. Keeping this distinction explicit lets the core reuse its tuned VLSU and FMA sequence.

B. Scalar Datapath and Schedule

The Scalar SMU implements the recurrence with a compact scalar datapath. A comparator selects m_{new} and forms two deltas; two EXP approximation/LUT blocks transform those deltas into scaled factors. Fixed-point multipliers combine the factors with l_{old} and l_{tile}, an accumulator forms l_{new}, and a reciprocal approximation normalizes both scaled lengths. Weight generation then emits w_{old} and w_{tile} for TCDM writeback. The RTL places these operations in the scalar states. In mode 1, control exits after scalar writeback and never enters the engine's `UPDATE_VECTOR` state; Proposed performs the vector update on existing RVV. All intermediate values remain on the scalar path until writeback, so the datapath follows the recurrence's data dependencies. The resulting interface exports weights as ordinary TCDM state for the next phase.

RTL observer counts establish a fixed per-row scalar schedule: `LOAD_SCALAR` consumes 13 cycles, `COMPUTE_SCALAR` 1, `COMPUTE_WEIGHT` 1, and `STORE_SCALAR` 9, for 24 SMU-busy cycles per row. LOAD and STORE account for 22 of 24 cycles as TCDM communication states; the two compute states occupy the remaining two cycles. The 24-cycle count describes the scalar-only FSM schedule. It is neither the fitted $C_s = 90.28$ nor software-visible or end-to-end latency, which also includes command sequencing and completion polling. The observer records only the four scalar states because mode 1 never enters `UPDATE_VECTOR`. The communication count reflects four reads plus four writes at state-machine handshake

granularity, while the compute states represent combinational evaluations.

C. TCDM-Mediated RVV Handoff

Mode 1 makes the handoff state explicit in TCDM. For each row, the Scalar SMU reads four FP32 scalar inputs, m_{old}, l_{old}, m_{tile}, and l_{tile}, from their TCDM arrays. It computes the new scalar state and writes m_{new}, l_{new}, w_{old}, and w_{tile} back to TCDM for all rows. The output weights are software-visible arrays rather than private accelerator state, which gives the core a stable handoff point. Because the weight arrays share TCDM with the state, the SMU does not retain a private weight interface for the core. The same row indexing keeps scalar outputs aligned with subsequent vector rows.

Software sequences the two engines through status and memory. It programs the MMIO command, starts mode 1, polls the status until done, and then reads the two weight arrays from TCDM. After completion, software invokes existing RVV to load $O_{\mathrm{old}}[D]$ and $O_{\mathrm{tile}}[D]$, apply the weights, and store $O_{\mathrm{new}}[D]$. The ordering gives RVV a completed, software-addressable weight state; no direct SMU-to-RVV path or weight-register bypass exists. Software remains the sequencing agent: it observes completion before exposing weights to RVV, so the core never consumes partially written rows. The RVV routine retains its existing load, multiply, FMA, and store sequence.

MMIO supplies command and status signals without changing the ISA. The cluster peripheral presents the software-selected operation to the Scalar SMU, while the engine contributes an independent TCDM requester to the shared interconnect. Existing core ports continue to serve RVV loads and stores, so the handoff traverses shared TCDM rather than a private engine link. This structure changes neither instruction decoding nor the RVV datapath. The interconnect arbitrates both requester classes through existing TCDM machinery, while MMIO remains the control-plane boundary. This composition adds no decoder instruction or custom vector operand.

This boundary fixes the architecture evaluated next: Scalar SMU removes state-dependent scalar recurrence work, and existing RVV retains the regular vector update through a software-mediated TCDM handoff. The Evaluation section measures the resulting cycle behavior and then separates scaling, model-derived configurations, and standalone hardware evidence. This framing lets the next section attribute cycle changes to the scalar recurrence while retaining a fixed vector implementation in the primary comparison. It also keeps standalone evidence separate from measured cycles.

III. Evaluation

Selective scalar offloading retains the low vector cost of existing RVV while moving the state-dependent recurrence to Proposed. Proposed combines Scalar SMU with existing RVV, B2R is the primary strong baseline, and Full is the Full-Offload Ablation. Across the three model-derived configurations, Proposed reaches a $4.81\times$ geometric-mean speedup in measured RTL/Verilator cycles over B2R. Speedup is B2R cycles divided by Proposed cycles.

A. Experimental Setup

We evaluate FP32 merge state on a Spatz cluster with (m, l, O) arrays resident in TCDM. Every cycle count comes from a measured RTL/Verilator run, so the comparison follows the implemented software-visible path rather than a model-only estimate. We apply the same correctness gate to every role and retain only passing outputs. B1 runs the scalar software context; B2R runs the software scalar recurrence and existing RVV vector update; Proposed assigns the recurrence to Scalar SMU and keeps existing RVV; Full runs the Full-Offload Ablation with both recurrence and vector update in the SMU. This role set isolates scalar offloading against a strong RVV baseline while retaining B1 as context and Full as a diagnostic comparison.

Standalone hardware evidence uses Yosys/ABC with the Nangate45 library. The area flow uses `abc -fast` to report mapped cells and Liberty cell-area units. A separate timing-driven flow reports critical delay and estimated standalone Fmax. Timing-flow cells and area remain outside the formal area comparison, because the two mappings answer different questions. The records provide no cluster PPA or power measurement. Formal cells and mapped area answer the standalone size question, while delay and Fmax report timing under separate constraints. This evidence describes the SMU-local implementation rather than a complete cluster.

B. Progressive Baseline

B1 to B2R isolates the contribution of RVV vectorization while B2R remains the primary strong baseline. B1 supplies scalar context but no primary speedup. B2R performs the scalar recurrence in software and the regular $O[D]$ update on existing RVV. Proposed moves only the state-dependent recurrence to Scalar SMU. Full is the Full-Offload Ablation and moves the vector update as well. Every reported speedup is B2R cycles divided by Proposed cycles, so the primary ratio attributes the gain to selective scalar offloading while holding the regular vector path in both roles.

C. Scaling and Cost Decomposition

We fit $C = C_0 + C_s N + C_v N D$ to measured RTL/Verilator cycles for each design. N is rows, D is vector dimension, C_0 is fixed work, C_s is fitted per-row scalar recurrence cost, and C_v is fitted per-element vector cost. Figure 2 displays these trends with a logarithmic C_s axis. The fit describes growth across measured shapes; it is distinct from the SMU-busy schedule and does not identify a single FSM counter. This view ties the primary comparison to scalar dependence without changing the regular vector path.

B2R fits $C_s/C_v = 1594.23/2.09$, whereas Proposed fits $90.28/2.13$. The fitted $C_s = 90.28$ is not the 24-cycle SMU-busy schedule. Proposed therefore cuts the fitted scalar term by $17.66\times$ while increasing the fitted vector term by 1.88%. The near-constant C_v shows that Scalar SMU offload removes recurrence work after RVV vectorization without changing the existing vector path. This decomposition ties the measured speedup to the scalar work moved off the core.

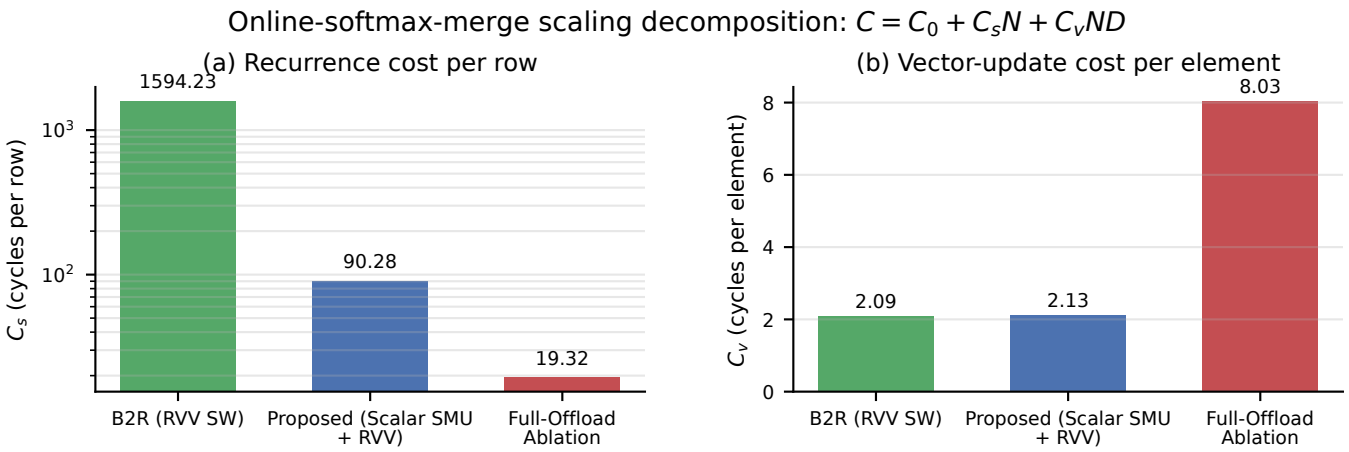

Fig. 2. Fitted C_s and C_v from RTL/Verilator cycles for B2R, Proposed, and the Full-Offload Ablation. Coefficients are fitted trends, not FSM busy cycles; C_s uses a logarithmic axis.

The Full-Offload Ablation fits $C_s/C_v = 19.32/8.03$, and its C_v is 3.78× Proposed. It therefore lowers the scalar term further but incurs a larger vector term when it absorbs the regular update. Proposed keeps both the scalar reduction and the lower vector cost, which defines the selective-offload trade-off measured in the primary comparison. The ablation therefore isolates the penalty of assigning regular elementwise work to dedicated hardware.

D. Model-Derived Configurations

We evaluate three model-derived attention-position shapes: BERT $(N, D) = (12, 64)$, Mistral $(32, 128)$, and Qwen14B $(40, 128)$. Table I reports measured RTL/Verilator merge-microbenchmark cycles for B1, B2R, Proposed, and the Full-Offload Ablation. The table records one-position merge work, with B1 retained as context; it does not represent end-to-end inference. This scope lets the cycle columns compare the same recurrence and regular vector update roles. Each role is measured on the one-position problem, while B1 remains separate from the primary speedup comparison.

Proposed/B2R speedups are 5.04× for BERT, 4.62× for Mistral, and 4.77× for Qwen14B, with 4.81× geometric mean. The three results support the same selective-offload conclusion across model-derived configurations: Proposed moves scalar recurrence work to Scalar SMU while B2R and Proposed retain the existing RVV update. The geomean summarizes measured ratios rather than fitted coefficients, giving a model-derived check on selective scalar offloading.

E. Hardware Cost and Full-Offload Ablation

Table II separates area and timing evidence. The area-oriented Yosys/ABC flow reports formal mapped cells and Liberty cell-area units with `abc -fast`; a separate timing-driven mapping reports critical delay and estimated standalone Fmax. Timing-flow area and cells do not enter the formal area comparison. Fmax is a pre-layout standalone estimate.

Proposed maps to 73,505 cells and 77,103.3 Liberty units, with 12.34 ns critical delay and 81.02 MHz estimated standalone Fmax. The Full-Offload Ablation maps to 107,372 cells and 114,713.3 Liberty units, with 13.20 ns and 75.74 MHz. Its standalone mapped area is 48.8% larger than Proposed. These values are standalone evidence, not cluster-level PPA. Full is larger than Proposed, has a longer critical delay, and has a lower estimated standalone Fmax; mapped area and timing remain results from separate flows.

Figure 3 places B2R at 1.00× measured-cycle speedup with zero SMU increment, Proposed at 4.81× and 77.103 k Liberty units, and the Full-Offload Ablation at 2.00× and 114.713 k units. The zero B2R increment denotes the software baseline, not zero cluster area. The speedup is frequency-independent, and circles mark geomeans while crosses mark workloads. The points connect measured speed to the standalone hardware view.

Cluster PPA, power, and energy remain unavailable, so these standalone mappings do not imply system-level area or throughput. The evidence supports a narrow conclusion: Proposed concentrates dedicated hardware on the scalar recurrence and reuses existing RVV resources for regular elementwise work, while the Full-Offload Ablation exposes the cost of extending offload into the vector path. This keeps the evaluation within measured cycles and standalone evidence rather than inferring cluster-wide resource use from mapped SMU values.

IV. Related Work

Dedicated Softmax and attention accelerators offload broader datapaths. Softermax combines online normalization with unnormalized and normalization hardware, while TEA-S

TABLE I
MODEL-DERIVED ONE-POSITION SHAPES AND MEASURED RTL/VERILATOR CYCLES. B1 PROVIDES CONTEXT; SPEEDUP IS B2R CYCLES DIVIDED BY PROPOSED CYCLES. VALUES ARE MERGE-MICROBENCHMARK CYCLES, NOT END-TO-END INFERENCE.

Workload	(N, D)	B1 context	B2R	Proposed	Full ablation	B2R/Proposed
BERT	$(12, 64)$	271,718	20,785	4,128	7,704	5.04×
Mistral	$(32, 128)$	1,401,702	59,500	12,866	34,728	4.62×
Qwen14B	$(40, 128)$	1,780,547	75,027	15,733	43,206	4.77×
Geomean	–	–	–	–	–	4.81×

TABLE II
STANDALONE MAPPED HARDWARE. CELLS/AREA AND DELAY/FMAX USE SEPARATE MAPPINGS; TIMING-FLOW AREA DOES NOT ENTER THE FORMAL AREA COMPARISON. FMAX IS A PRE-LAYOUT STANDALONE ESTIMATE.

Design	Mapped cells (area flow)	Mapped area (Liberty units)	Critical delay (timing flow)	Est. standalone Fmax
Proposed	73,505	77,103.3	12.34 ns	81.02 MHz
Full-Offload Ablation	107,372	114,713.3	13.20 ns	75.74 MHz

implements a compact PLAC-based Softmax architecture [4], [5]. SoftEx couples a full-Softmax HWPE to shared TCDM [6]. Alexandridis et al. place recurrence-related computation and the output-vector update in custom FlashAttention hardware [7]. These designs therefore retain more vector work in dedicated hardware than Proposed.

Programmable approaches keep Softmax on the processor side. Spatz provides the RVV/TCDM substrate used here [3]. VEXP adds BF16 scalar/SIMD exponential instructions while software performs the remaining partial Softmax [8]; Titopoulos et al. implement FlashAttention entirely with standard RVV instructions [9]. Among the reviewed designs, we did not identify an architecture that dedicates hardware only to the state-dependent online-merge scalar recurrence and then reuses an existing RVV datapath through a TCDM-mediated weight handoff for regular $O[D]$ work. To our knowledge, this selective-offload boundary differs from both full-offload and programmable-vector approaches.

V. CONCLUSION

This work defines and validates a selective specialization boundary for online-softmax merge. Proposed assigns the state-dependent scalar recurrence to a cluster-local Scalar SMU and retains the regular $O[D]$ update on the existing RVV datapath through TCDM. Across three model-derived configurations, RTL/Verilator measurements show a 4.81× geometric-mean measured-cycle speedup over B2R, while the fitted decomposition preserves B2R-like vector cost. The Full-Offload Ablation reduces the scalar term further but increases vector cost and standalone mapped SMU area relative to Proposed. Selective Offloading therefore concentrates dedicated hardware on the recurrence and reuses existing RVV resources for regular elementwise work.

REFERENCES

[1] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," in *Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 16 344–16 359.

[3] M. Perotti, S. Riedel, M. Cavalcante, and L. Benini, "Spatz: Clustering compact risc-v-based vector units to maximize computing efficiency," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 7, pp. 2488–2502, 2025.

[4] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, "Softermax: Hardware/software co-design of an efficient softmax for transformers," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, 2021, pp. 469–474.

[5] Z. Mei, H. Dong, Y. Wang, and H. Pan, "TEA-S: A tiny and efficient architecture for PLAC-based softmax in transformers," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 9, pp. 3594–3598, 2023.

[6] A. Belano, Y. Tortorella, A. Garofalo, L. Benini, D. Rossi, and F. Conti, "A flexible template for edge generative ai with high-accuracy accelerated softmax and gelu," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 15, no. 2, pp. 200–216, 2025.

[7] K. Alexandridis, V. Titopoulos, and G. Dimitrakopoulos, "Low-cost flashattention with fused exponential and multiplication hardware operators," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, 2025, pp. 1–6.

[8] R. Wang, G. Islamoglu, A. Belano, V. Potocnik, F. Conti, A. Garofalo, and L. Benini, "Vexp: A low-cost risc-v isa extension for accelerated softmax computation in transformers," in *Proc. IEEE 32nd Symp. Comput. Arithmetic (ARITH)*, 2025, pp. 37–44.

[9] V. Titopoulos, K. Alexandridis, and G. Dimitrakopoulos, "Vectorized flashattention with low-cost exponential computation in risc-v vector processors," *J. Supercomput.*, vol. 82, no. 4, 2026.

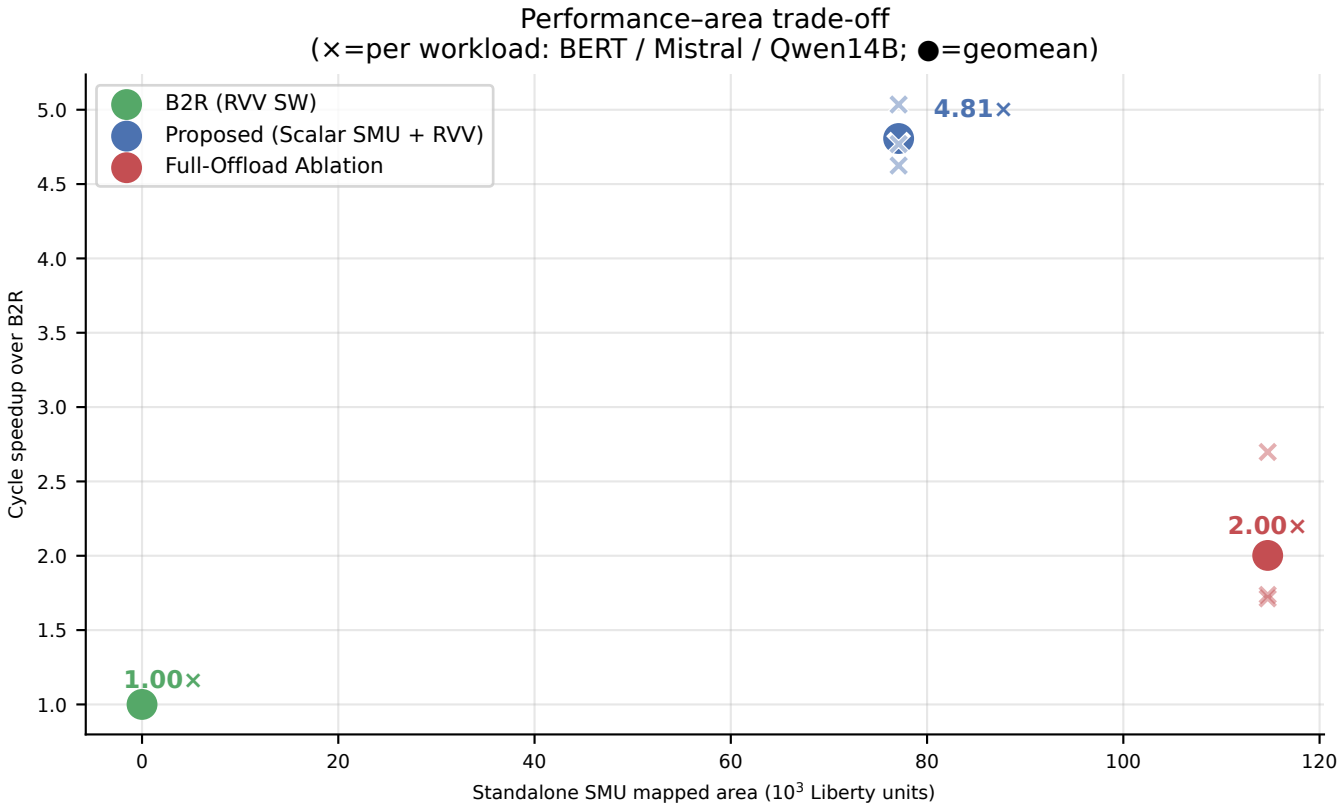

Fig. 3. Frequency-independent measured-cycle speedup over B2R versus standalone SMU mapped area. B2R's zero SMU increment is not zero cluster area; circles mark geomeans and crosses mark workloads.